



Coleções

Aula 09 · Bloco 3 — POO na Prática

Estruturas que crescem, encolhem e se organizam sozinhas

Nesta aula

1. Onde o **array** não dá conta
2. `List` e `ArrayList` — e os **generics**
3. Buscar objetos: a dependência do `equals`
4. `Map` — quando existe uma **chave**
5. `Set` — sem repetição

Os três problemas do array

```
Aluno[] turma = new Aluno[30];
```

- ✂ **Tamanho fixo.** Chegou o aluno 31? Criar array maior e copiar tudo na mão;
- ☪ **Buracos.** Removeu a posição 5? Fica um `null` no meio, e todo laço precisa desviar;
- 📄 **Contagem manual.** `turma.length` é 30 desde o início, com 3 alunos cadastrados.

List e ArrayList

```
List<Aluno> turma = new ArrayList<>();
```

Duas ideias importantes nessa linha:

- **<Aluno>** é o **generic** — a lista só aceita alunos. `turma.add("texto")` não compila;
- **List** à esquerda, **ArrayList** à direita — você declara pelo **contrato** e escolhe a **implementação**. Trocar para **LinkedList** amanhã é mudar uma palavra.

💡 É a lição da aula 08 aplicada dentro da biblioteca padrão.

⚠ Generics não aceitam primitivos

`List<int>` não existe.

É `List<Integer>`, `List<Double>`.

As classes *wrapper* embrulham os primitivos —
e a conversão é automática: `numeros.add(42)`
funciona.

As operações do dia a dia

```
nomes.add("Ana");           // adiciona no fim
nomes.add(1, "Beto");        // insere na posição, empurrando o resto

nomes.size();               // MÉTODO – o array usa .length
nomes.get(0);               // get(i) – o array usa [i]
nomes.contains("Ana");
nomes.indexOf("Carla");     // -1 se não existir

nomes.set(0, "Ana Paula");  // substitui
nomes.remove("Beto");       // pelo objeto
nomes.remove(0);            // pela posição
```

E `System.out.println(nomes)` já imprime bonito, de graça.

⚠️ Nunca remova dentro de um **for-each**

`ConcurrentModificationException`

Para remover em bloco:

```
lista.removeIf(a -> a.getMedia() < 5);
```

(a sintaxe da seta é assunto da aula 14)

Buscando objetos: o padrão

```
public Aluno buscarPorMatricula(String matricula) {  
    for (Aluno aluno : turma) {  
        if (aluno.getMatricula().equals(matricula)) return aluno;  
    }  
    return null;           // não achou – e quem chama TEM de tratar  
}
```

💡 Devolver `null` é a fonte nº 1 de `NullPointerException`. Na aula 10 você aprende a **lançar exceção**; na 14, a devolver `Optional`.

Isto falha

```
turma.add(new Aluno("Ana", "1001"));  
Aluno procurado = new Aluno("Ana", "1001");  
  
turma.contains(procurado); // false
```

`contains()` usa `equals()` internamente.

Sem sobrescrita, o `equals` herdado compara
referências.



Toda classe que vai morar numa coleção precisa de `equals` e `hashCode` .

Sobrescreva os dois, como na aula 07, e o mesmo código passa a devolver `true` .

Map : quando existe uma chave

Lista é boa para "todos os alunos". Mas para "me dê o aluno 1001", percorrer tudo é desperdício.

```
Map<String, Aluno> porMatricula = new HashMap<>();

porMatricula.put("1001", new Aluno("Ana", "1001")); // chave → valor
Aluno ana = porMatricula.get("1001");              // busca DIRETA
porMatricula.get("9999");                           // null se não existir

porMatricula.containsKey("1001");
porMatricula.remove("1002");
```

Percorrendo um **Map**

```
for (String chave : porMatricula.keySet()) { ... }

for (Map.Entry<String, Aluno> e : porMatricula.entrySet()) {
    System.out.println(e.getKey() + " → " + e.getValue());
}
```

Chaves são únicas — **put** repetido **substitui**. E **sem ordem garantida**: se importar, use **LinkedHashMap** ou **TreeMap**.

💡 É aqui que o **hashCode()** da aula 07 aparece: o **HashMap** usa o hash para achar a "gaveta" certa quase instantaneamente.

Set : coleção sem repetição

```
Set<String> cursos = new HashSet<>();  
cursos.add("Java");  
cursos.add("Python");  
cursos.add("Java");           // ignorado: já existe  
  
cursos.size();                // 2
```

Use quando repetição não faz sentido: CPFs cadastrados, tags, códigos únicos.

Assim como no **Map**, a unicidade é decidida por **equals** / **hashCode**.

Qual usar

Estrutura	Use quando	Cuidado
<code>List</code> / <code>ArrayList</code>	ordem importa, repetição permitida	busca percorre tudo
<code>Map</code> / <code>HashMap</code>	existe chave natural (código, CPF)	sem ordem garantida
<code>Set</code> / <code>HashSet</code>	elementos únicos, ordem não importa	sem índice, sem <code>get(i)</code>

Exercícios da aula

Na pasta `aula-09/` :

1. `ListaDeCompras.java` — leitura até `fim` , lista numerada, remoção pelo nome;
2. `Turma` + `Aluno` — `List` **privada**; nenhum `main` toca na lista direto;
3. `Igualdade.java` — `contains()` antes e depois de sobrescrever `equals` ;
4. `Agenda.java` — `Map<String, String>` com menu completo;
5. **Desafio** 🌶️ `Estoque.java` — `Map` por código + `Set` de categorias.

Próxima aula

Aula 10 — Exceções

Impedir que o programa quebre
na mão do usuário.